

# Guía Práctica de Arquitectura de Software

Alumnas: Humpiri Mariana y Daisi Flores

Fecha: 10-06-25

## Ejercicio 1: El Proyecto Personal

Análisis de Re

### Funcionales:

- Crear, editar y eliminar artículos.
- Subir imágenes.
- Mostrar un portafolio de proyectos.

### No Funcionales:

- Bajo Costo.
- Time-to-Market.
- Simplicidad de Mantenimiento.

### Drivers Arquitectónicos

1. Simplicidad de Mantenimiento.
2. Bajo Costo.
3. Time-to-Market.

◇ Patrón Propuesto: Arquitectura Monolítica en Capas

### Justificación

- Despliegue y desarrollo rápido.
- Bajo costo operativo.
- Más que suficiente para una aplicación personal.

### Diagrama de Alto Nivel

Usuario

↓

[ Interfaz Web (HTML/CSS/JS) ]

↓  
[ API REST (Express.js) ]  
↓  
[ Lógica de Negocio ]  
↓  
[ Acceso a Datos (ORM) ]  
↓  
[ Base de Datos (MongoDB Atlas o Firestore) ]

## Microservicios

No recomendados para este caso por su complejidad y costo.

## Estrategia Propuesta

### Despliegue Propuesto:

- Frontend: GitHub Pages o Netlify.
- Backend: Render o Vercel.
- DB: MongoDB Atlas o Firebase.

**CI/CD:** GitHub Actions para build/test/deploy automático.

### Seguridad:

- JWT para autenticación.
- Validaciones con express-validator.
- Subida segura con multer.

### Diagrama CI/CD:

GitHub (main) → GitHub Actions → Netlify (Frontend) / Render (Backend) → MongoDB Atlas / Firestore

## Ejercicio 2: La Startup en Crecimiento

### Análisis de Requisitos

#### Funcionales:

- Catálogo, carrito, pagos, usuarios.

#### No Funcionales:

- Escalabilidad.
- Evolucionabilidad.
- Resiliencia.

## Drivers Arquitectónicos

1. Escalabilidad.
2. Mantenibilidad.
3. Alta Disponibilidad.

◇ Patrón Propuesto: Microservicios + API Gateway

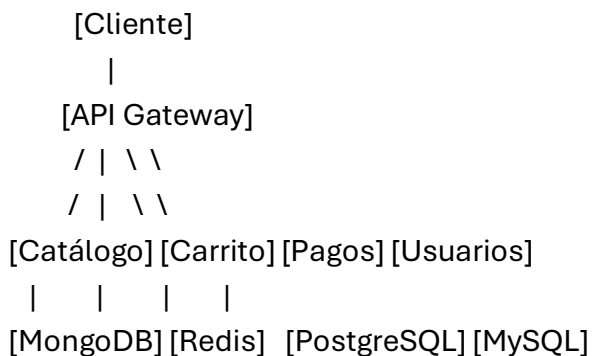
## Justificación

- Equipos paralelos pueden trabajar sin conflictos.
- Se escalan servicios críticos de forma independiente.
- Falla en recomendaciones no afecta pagos.

## Servicios Clave:

- Servicio Catálogo
- Servicio Carrito
- Servicio Pagos
- Servicio Usuarios

## Diagrama de Arquitectura



## Estrategia Propuesta

**CI/CD:** Deploy automatizado por microservicio.

**Mensajería interna:** Kafka o RabbitMQ para comunicación asincrónica.

**Resiliencia:** Retry, circuit breaker, health checks con Kubernetes.

**Base de Datos por Servicio:** Aisladas para evitar acoplamiento.

**Trade-off:** Alta complejidad DevOps.

## Ejercicio 3: Aplicación Social en Tiempo Real

### Análisis de Requisitos

#### Funcionales:

- Edición colaborativa.
- Notificaciones.
- Historial y panel de análisis.

#### No Funcionales:

- Baja latencia.
- Desacoplamiento.
- Resiliencia.

### Drivers Arquitectónicos

1. Tiempo Real.
2. Desacoplamiento.
3. Resiliencia.

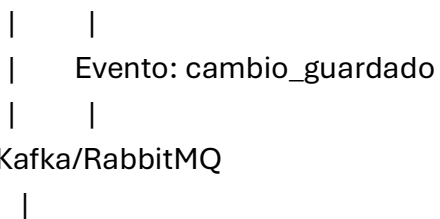
◇ Patrón Propuesto: Arquitectura Orientada a Eventos (EDA)

### Justificación

- Cada componente puede fallar sin afectar el resto.
- Mensajes asíncronos mantienen la experiencia fluida.

### Diagrama de Flujo

[Usuario] → [Editor Web] → [Servicio Documento]



[Servicio Notificaciones]

[Servicio Versionado]

[Servicio Analytics]

### Estrategia Propuesta

#### Tecnologías:

- WebSocket para edición colaborativa.
- Kafka como broker de eventos.
- MongoDB para versionado.
- Elasticsearch para análisis rápido.

### Trade-off:

- Mayor dificultad de trazabilidad.

## Ejercicio 4: Sistema Corporativo Universitario

### Análisis de Requisitos

#### Funcionales:

- Admisiones, Inscripciones, Finanzas, Biblioteca.

#### No Funcionales:

- Mantenibilidad.
- Alineación con dominios.
- Integración con sistemas legacy.

### Drivers Arquitectónicos

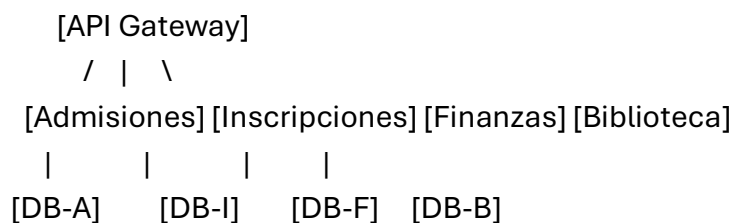
1. Mantenibilidad.
2. Modularidad por Dominio.
3. Interoperabilidad.

◇ Patrón Propuesto: Arquitectura por Dominios (DDD) + Microservicios

### Justificación

- Permite que cada departamento evolucione de forma independiente.
- Alinea estructura técnica con organizacional.

### Diagrama de Alto Nivel



### Gestión de Datos Compartidos

- Mediante eventos compartidos y sincronización por colas de mensajes.
- Servicio Central de Identidad de Estudiante (referenciado por todos).

### Estrategia Propuesta

**Mensajería:** Kafka o EventBridge para notificar eventos como "Alumno Inscripto".

**Gateway de Integración:** API REST con adaptadores para sistemas legacy.

**CI/CD:** Pipelines independientes por dominio.

**Documentación:** OpenAPI + Wikis por dominio.

**Trade-off:** Complejidad organizacional y técnica.